3

Newton School of Technology

OOPs Introduction

By: Aditya

Join the lecture online on your dashboard.

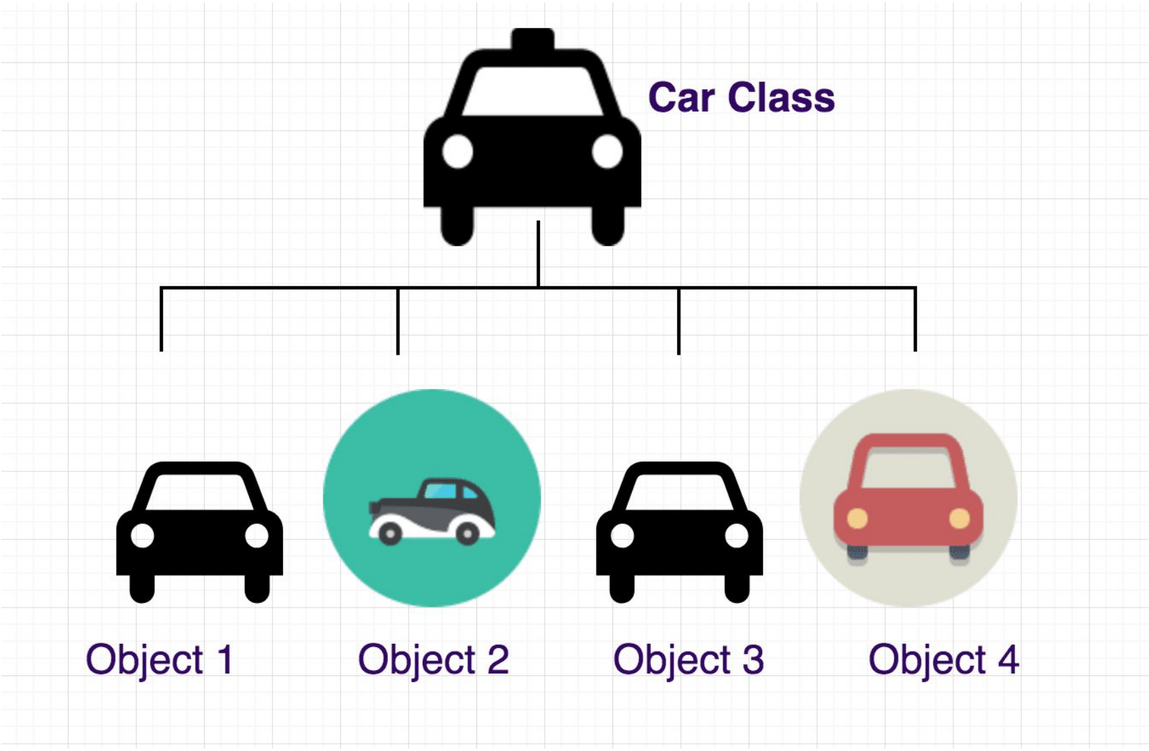
Recap of Previous Lecture!

Quick Recap :

(Classes & Objects → Systems)

- Last time, you met **Classes and Objects** — the tiny superheroes of software
- Every object had:
  - **State** → what it knows
  - **Behavior** → what it does
  - **Identity** → who it is
- You even drew **UML boxes** to show who does what

Congrats - you built the **LEGO bricks of programming**



But… here’s the real question

- What happens when thousands of LEGO bricks come together?

- How do objects collaborate, depend on each other, and follow rules?

That’s where **today’s story begins** - from objects to a software city

Software city == System

So,What Exactly Is a System?

- A **system** is not magic
- It's just a **bunch of objects working together** to get something done

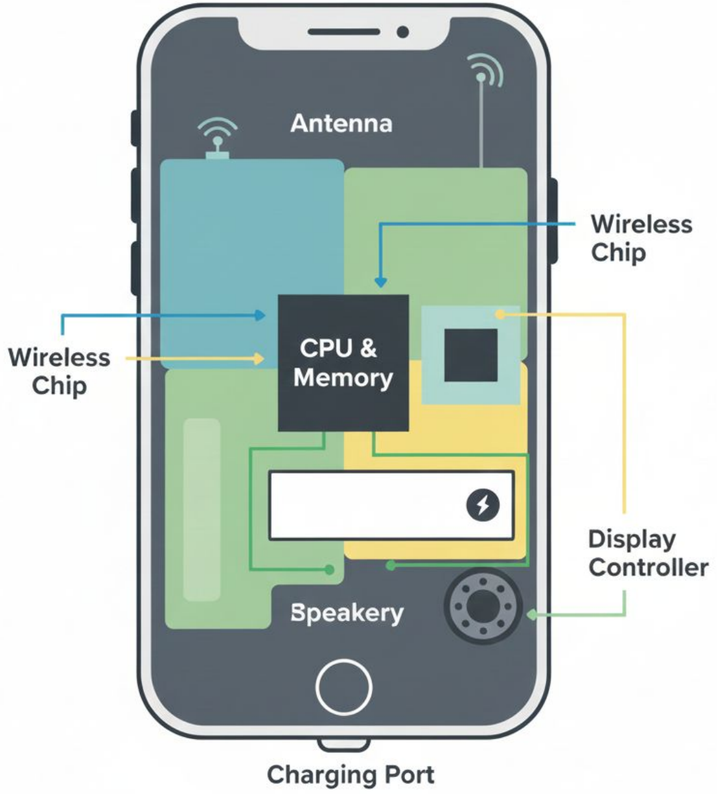
Think like this:

- Individually:
  - Each object does one useful job
- Together:
  - They form a **complete, functioning system**

Key idea to remember

Objects are good alone… but **powerful together**

That's how real-world software is built — not one object, but a **team of them**.



Meet Our Hero 👨‍💻 – Dev

- Meet **Dev** - a developer just like you
- Knows basics:
  - Classes
  - Objects
  - Methods & variables

Dev's problem:

- His programs work… but only when they're **small** 😐
- As soon as the app grows:
  - Code becomes messy
  - Changes break other parts
  - Fear of touching old code 😨

Dev asks the big question:

- How do professionals build BIG software without losing their mind?”
- This is where **OOP as a system-thinking tool** enters Dev’s life

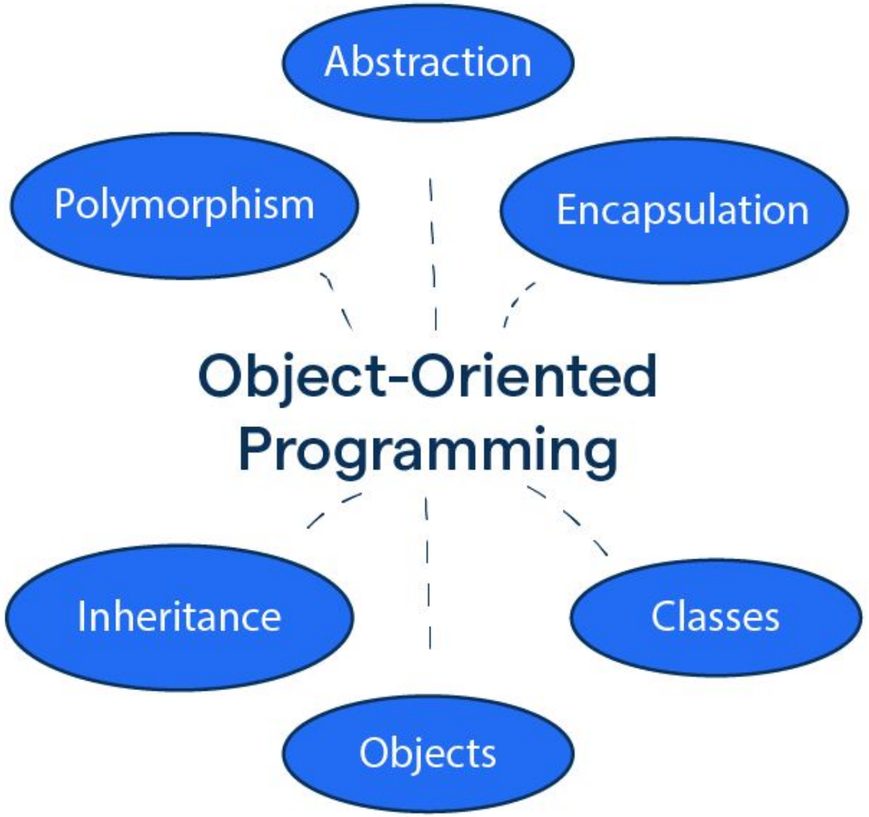
what is OOP ?

OOP = Object-Oriented Programming

- A programming paradigm based on the idea of objects
- An object contains:
  - Data (attributes) → what it has
  - Code (methods) → what it does

How OOP thinks differently:

- Focuses on data + behavior together
- Not just functions and logic floating around



Object Oriented Programming

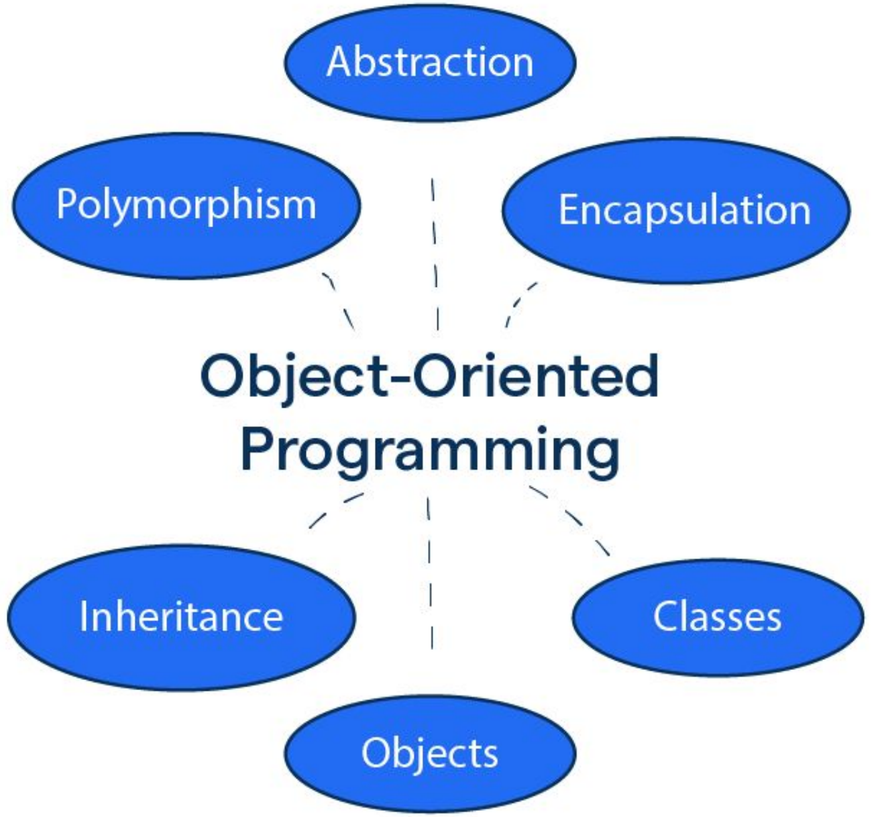
Why OOP feels real:

- Models **real-world entities** and their interactions
- Just like people, cars, accounts, or players

What OOP gives you

- Modular code
- Reusable components
- Easier maintenance when systems grow

In short: OOP helps you **think in objects**, not just write code.



Why OOP?

- As programs grow:
  - Code becomes **large** 📈
  - Logic becomes **complex** 🧩
  - Managing everything becomes a nightmare

Without OOP:

- One small change
- Ten files break
- Debugging feels like archaeology

OOP to the Rescue

- OOP breaks a big program into:
  - **Small, manageable objects**
- Each object has:
  - A **clear responsibility**

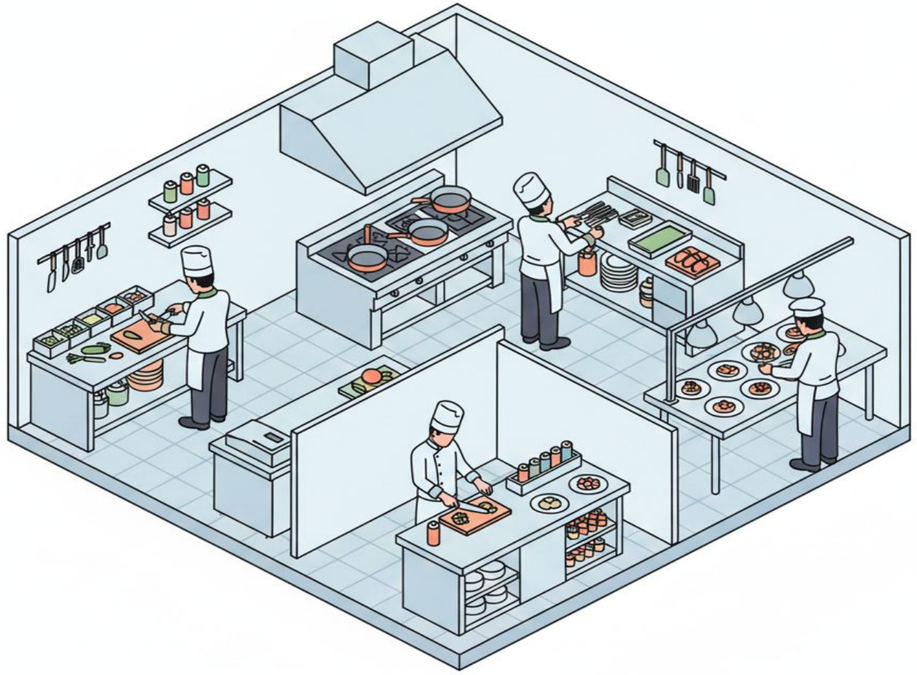
When requirements change:

- Modify **only the affected objects**
- Rest of the system stays safe 🛡️

Result:

- Fewer bugs
- Easier updates
- No fear of touching code

OOP doesn’t stop change - it **controls** change



How Programming Worked Before OOP

Procedural Programming is the classic style before OOP

Key points:

- Based on **procedure calls / functions**
- Programs = **sequence of instructions** executed step-by-step
- Focus: write procedures that operate on data
- Also called **Imperative Programming**
- Approach: **Top-down problem-solving**

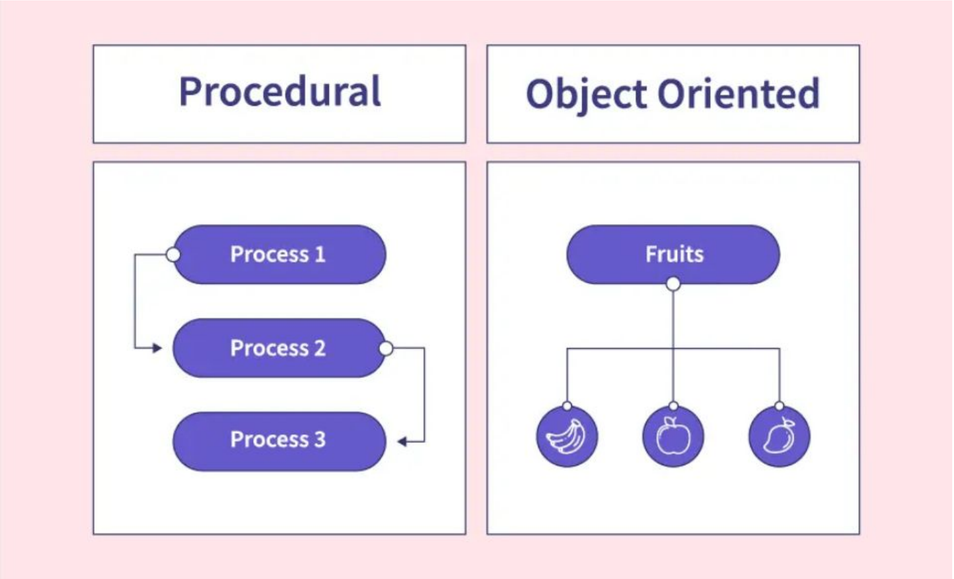
Examples: C, Pascal, Fortran, BASIC

Dev's realization:

- Works fine for small apps
- But big programs break easily 😵
- Hard to reuse code or model real-world entities

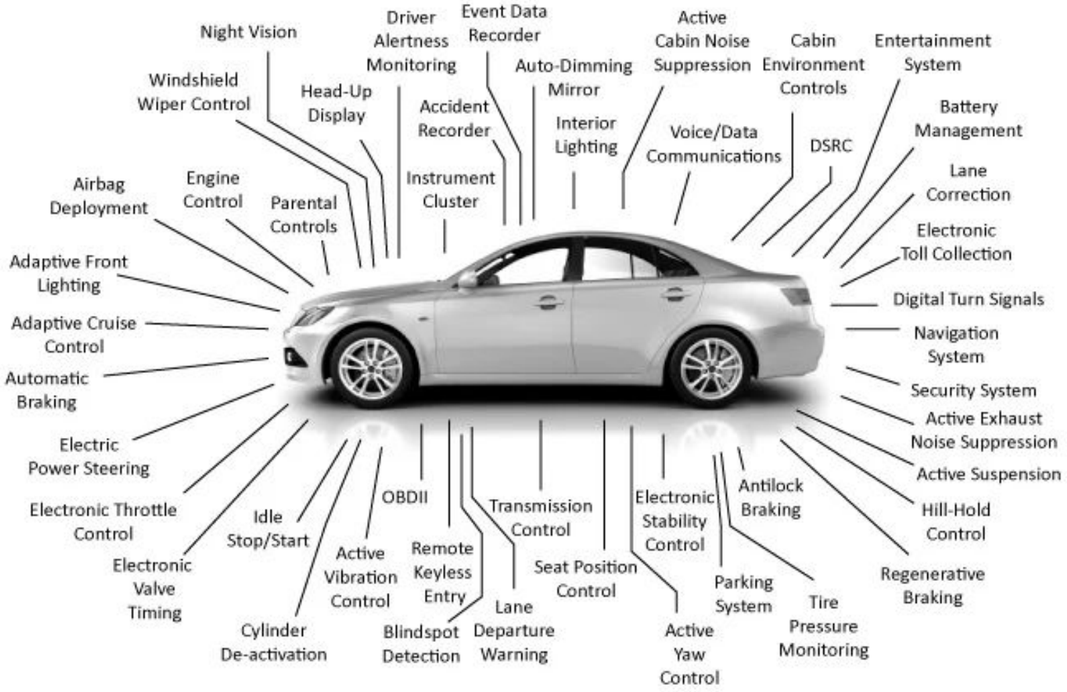
Procedural pain points:

- Code is **linear** and hard to manage
- Functions scattered, no clear ownership of data
- Hard to model real-world entities
- Changes can break unrelated code



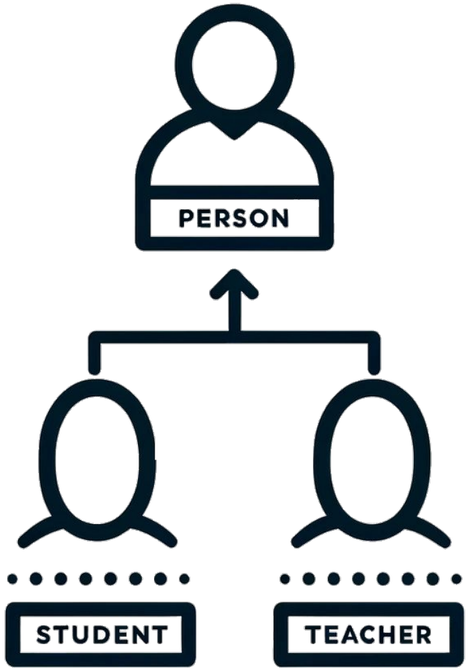
OOP advantages for Dev:

- Makes programs **more organized and manageable**
- Objects have **clear responsibilities**
- Changes affect only **relevant parts of code**
- Promotes **code reuse naturally**
- Better **real-world modeling** → think in entities, not just functions



Inheritance – A Shortcut! 🛠️

- Dev is building his system and notices **lots of repeated code** across classes
- Frustration sets in: “Do I have to copy-paste again?” 😩
- A mentor (or a friendly StackOverflow post 😉) whispers: **“Use inheritance, dummy. Save time, reuse code.”**

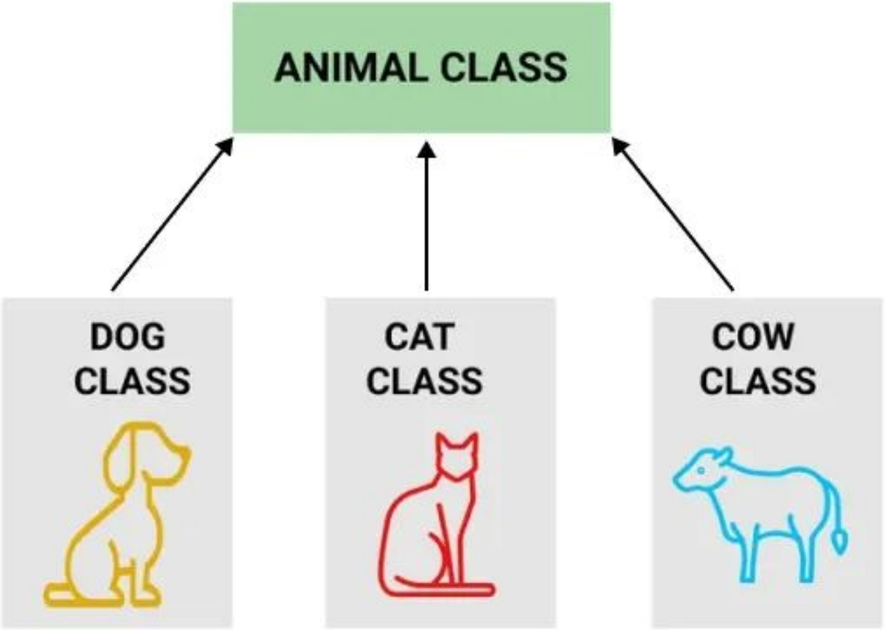


Inheritance (Is-A)

- Inheritance allows one class to inherit the properties and methods of another class. It represents an "IS-A" relationship where a subclass IS-A type of its superclass.

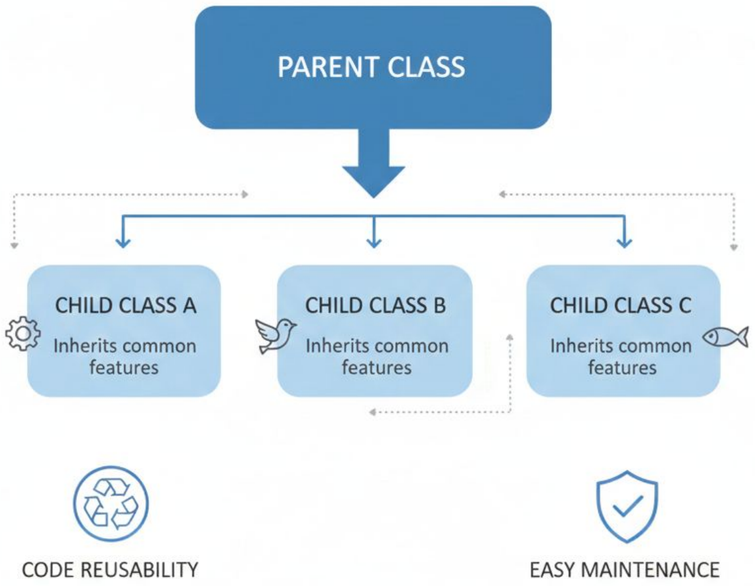
- **Example:** A Dog IS-A Animal, A Cat IS-A Animal

- Saves you from rewriting stuff multiple times.



Why Use Inheritance?

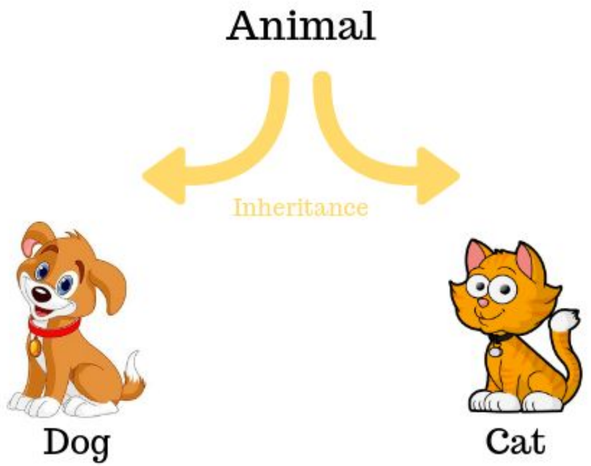
- **Code Reusability:** Avoid duplicating code across similar classes
- Different **child objects** can be treated as the same parent type, while still behaving differently at runtime.
- **Logical Hierarchy:** Model real-world relationships
- **Easier Maintenance:** Changes in parent class automatically flow to child classes



What is IS-A?

IS-A is the inheritance relationship test. If "X IS-A Y" makes logical sense, then X can inherit from Y.

Example: Dog IS-A Animal,Cat IS-A Animal



Key Points:-

Represents "kind of" or "type of" relationship

- A Dog is a *kind of* Animal, not just related to it

Enables code reusability through inheritance

- Dog automatically gets all Animal properties and methods

Uses the extends keyword

- class Dog extends Animal {}

When to use IS-A?

Subclass is a specialized version of parent

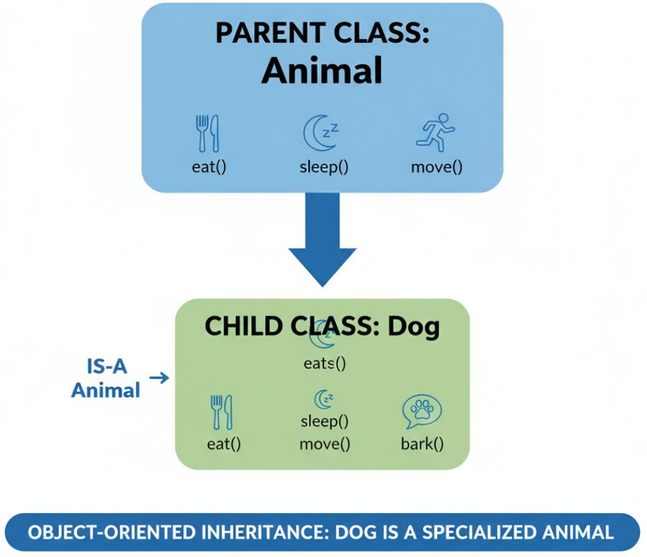
- Dog is a specific type of Animal with extra behaviors (bark)

Shared characteristics exist

- All Animals eat, sleep, move - Dog inherits these automatically

Relationship is permanent

- A Dog will always be an Animal, never changes



When not to use IS-A?

- When It's "HAS-A" relationship (Dog HAS-A collar, not IS-A collar) , what is Has - a ..? Wait for a while ..

- Just reusing code without true specialization

- Subclass doesn't fit parent's behavior (Penguin IS-A Bird but can't fly)

Common Pitfalls of Inheritance ⚠️

1. **Deep hierarchies**
   - Long chains of classes
   - Changing the base class can **wreck everything**

2. **Over-generalization**
   - Base classes trying to do **too much**
   - Confuses developers and users

Best practices:

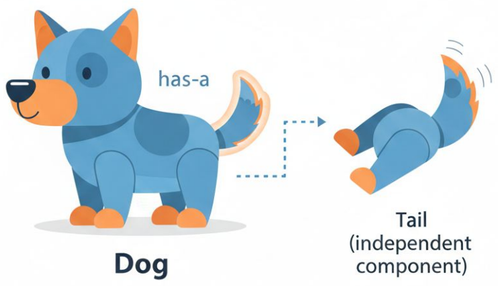
- Keep inheritance **simple and meaningful**
- Use inheritance only when a **true IS-A relationship** exists

Composition (Has-a)

- Composition is when a class contains or uses another class as a part.

- **Dog HAS-A Tail** - no need to make Tail a Dog, just let it wag itself.

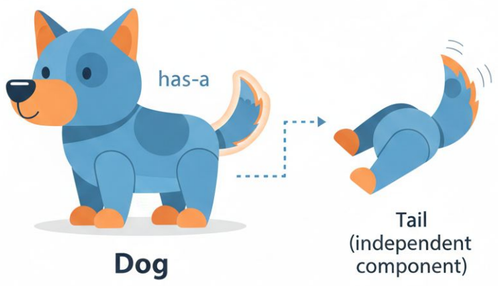
- Lets you reuse code without messy inheritance - mix and match parts freely.



Composition (Has-a)

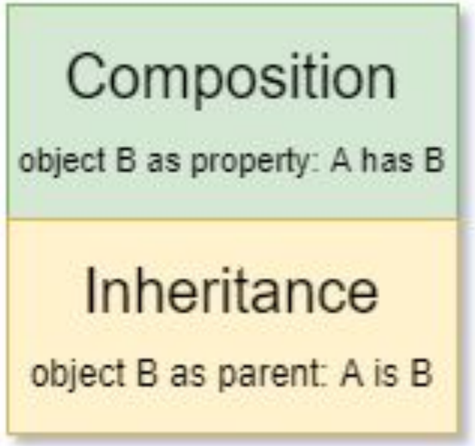
👉 Building a class by combining other classes, instead of becoming them.

A class **DOES NOT** become another class,
it simply **USES** it.



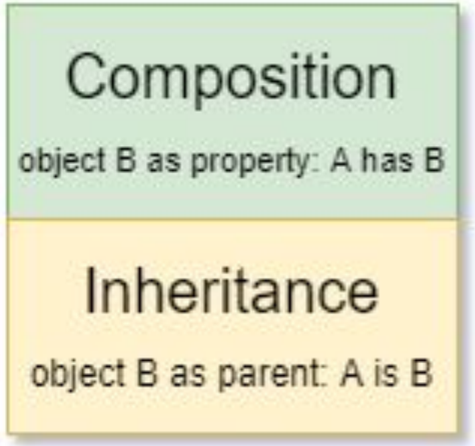
Is-a vs Has-a

Inheritance (Is-a)	Composition (Has-a)
“You are this thing”	“You have this thing”
`Dog extends Animal`	Dog has a Tail
Child gets parent DNA	Object borrows another object’s skills
One wrong change → many break	Change stays in one place
Feels powerful, gets risky	Feels boring, stays safe
Use when relationship is obvious	Use when relationship feels forced



Is-a vs Has-a

- A Dog _____ Animal.
- A Car _____ Engine.
- A Smartphone _____ Camera.
- A Bicycle _____ Vehicle.
- A VideoPlayer _____ SubtitleFeature.



Can you think of a real-world coding scenario where using inheritance would be a poor design choice, and why?

Coding Activity

Create the base class `Vehicle` and the child class `Bicycle`. Then, design `ElectricBicycle`.

Create the base class Vehicle and the child class Bicycle. Then, design ElectricBicycle.

```
class Vehicle {
  constructor(public wheels: number) {}
}
class Bicycle extends Vehicle {
  constructor() {
    // Calls the parent constructor
    super(2);
  }
}class ElectricBicycle extends Bicycle {
  constructor(public batteryLevel: number)
{   super();
  }
}
```

Is-a vs Has-a …. Did Inheritance Really Fit?

Check if ElectricBicycle truly is-a Bicycle, or if giving it a Battery (has-a) would make the design clearer.

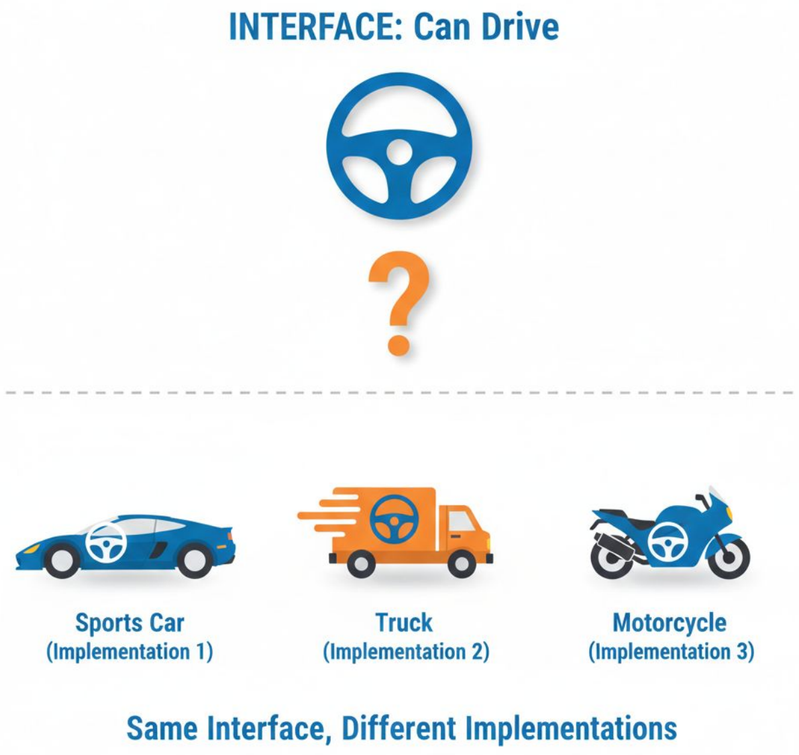
Interface

- An interface is a contract, not an implementation.
- It says what a class can do, not how it does it.
- Think: “You can drive” - I don’t care how you drive.
- Uses interface keyword in typescript

```
interface Car { name: string; drive(): void; }
```

- Classes implement interfaces using implements keyword

```
class Dog implements Pet {
name: string;
play(): void {
console.log("Fetching ball!"); } }
```

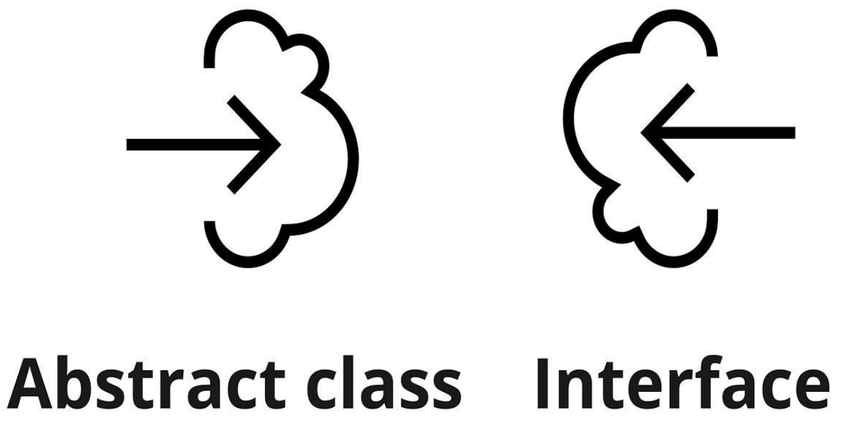


Why Interfaces? (Loose Coupling)

- Code depends on what an object does, not what it is.
- Swap implementations without touching main code.
- Less coupling = fewer “why did this break?” moments.

Interface vs Abstract Class

- Interface = what you must do.
- Abstract class = what you are + some default behavior.
- Interfaces solve design problems, abstract class solve code reuse.



(**Rule of thumb**: Need flexibility? Interface. Need shared code? Abstract class.)

Interface vs Abstract Class Example

Interface: Payable (What you must do)

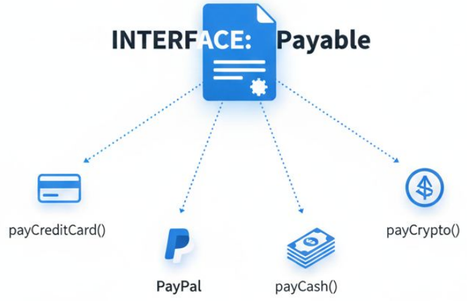
- CreditCard implements Payable
- PayPal implements Payable
- Cash implements Payable
- Bitcoin implements Payable

All must have: processPayment() and validatePayment()

Abstract Class: BankPayment (What you are + reusable code)

- DebitCard extends BankPayment
- DirectDebit extends BankPayment

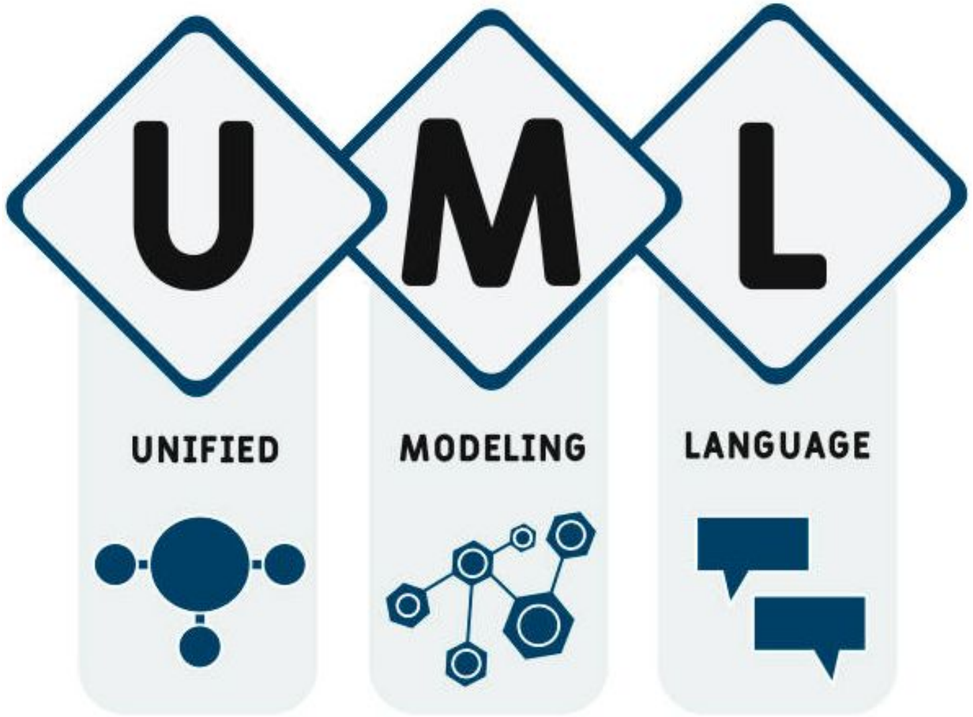
All inherit: verifyBankDetails() and logTransaction()



UML & TypeScript in Action

- Inheritance (Generalization): solid line → parent class.

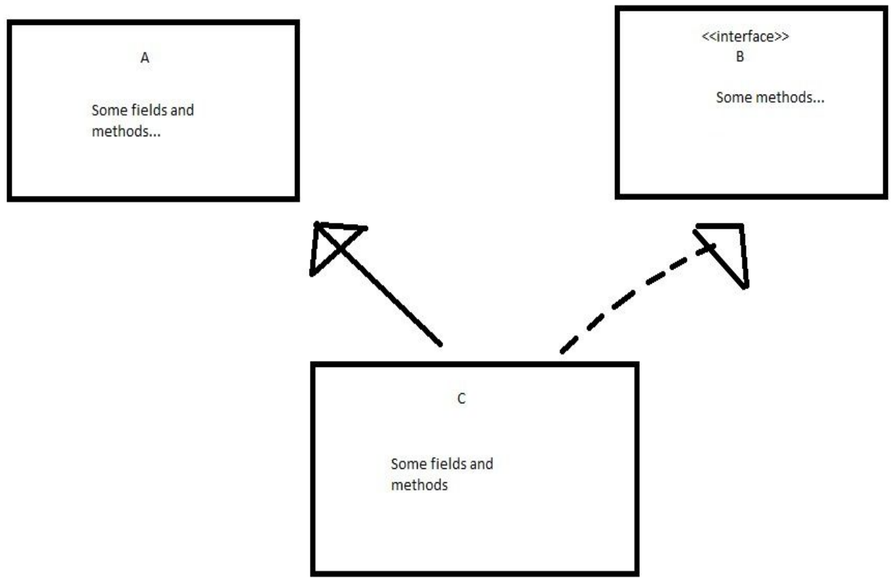
- Interface (Realization): dashed line → interface.



(**Key idea**: UML shows relationships, TypeScript enforces contracts.)

UML Notation: Inheritance vs Interface

- Generalization (Inheritance): solid line → parent class >> ("I am you.")
- Realization (Interface): dashed line → interface ("I promise I can do this.")
- Interfaces reduce coupling because code depends on behavior, not concrete classes.



Result: flexible design, easier changes, fewer breakages.

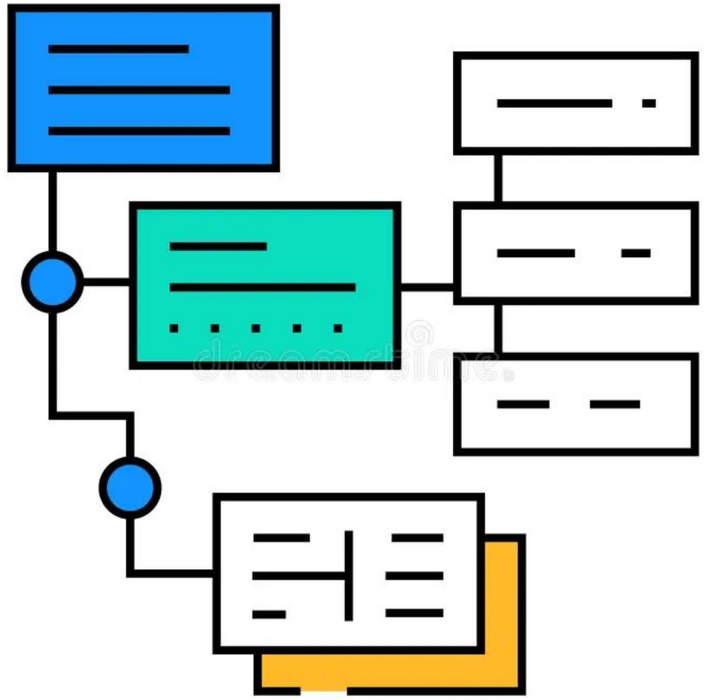
TypeScript Example

```
interface Drivable {
  drive(): void;
  getSpeed(): number;
}
class Car implements Drivable {
  drive() {
    console.log("Car is
moving");
  }
  getSpeed() {
    return 60;
  }
}
```

MCQ Time ..

UML Deep Dive (Practice Time ..)

- Time to draw, not just nod.
- Animal → Dog using generalization (solid arrow to parent).
- Add Pettable interface → Dog uses realization (dashed arrow).
- One class, two relationships — welcome to real design.



(**Brain shift:** code → picture.)

Your Turn: Draw the UML

- Draw the UML for **Animal** → **Dog** using a solid inheritance arrow.
- Add the **Pettable** interface and connect it to **Dog** with a dashed arrow.
- Check arrow direction carefully - it matters.

What bad UML looks like (e.g., drawing the relationship Engine extends Car), and why that design is structurally incorrect.

Quick Quiz + Open QnA

- Define inheritance in your own words.
- In programming terms, why do interfaces reduce coupling?
- Draw the UML notation where a class called Phone implements an interface called Connectable.

Summary & Takeaways

- OOP is your tool for managing complexity by introducing modularity.
- Inheritance is powerful but leads to rigidity if the "Is-a" relationship is weak.
- **Composition** is often the more flexible and durable design choice ("Has-a").
- Interfaces enforce contracts and encourage loose coupling.
- **UML:** Remember your arrows-solid for generalization, dashed for realization!

Your Turn

Try It Out ..

- **Read:** Review TypeScript classes, interfaces, and inheritance; revise core OOP concepts.
- **Practice:** Model a simple game entity using composition and draw UML diagrams for today's examples.

Project

Smart Home Device Model

- Build a small OOP system in TypeScript.
- Create a Connectable interface and implement it in SmartBulb, SmartThermostat, SmartSpeaker.
- Add features using composition (e.g., VoiceControl, MotionSensor).
- Draw a UML diagram showing inheritance and interface realization.

Newton School of Technology

Thank You ..